Contrôle d'un Jeu Vidéo par Gestes de la Main

Vision par Ordinateur — Cours d'Initiation
Master 2 Mathématiques

Webcam $\longrightarrow$ MediaPipe $\longrightarrow$ CNN $\longrightarrow$ WebSocket $\longrightarrow$ Jeu

Auteurs Christ Bohou
Djafarou Oulare

Date Mars 2026

Ce rapport décrit le pipeline complet de reconnaissance de gestes en temps réel appliqué au jeu Space Invaders, depuis la collecte des données jusqu'à l'intégration dans le navigateur.

Table des matières

1 Introduction et Objectif **3**

2 Architecture Globale du Système **3**
2.1 Vue d'ensemble 3
2.2 Rôle de chaque composant 4

3 Collecte et Préparation des Données **4**
3.1 Approche 1 : images brutes (64×64 pixels) 4
3.2 Approche 2 : landmarks normalisés (vecteur 63D) 4
3.3 Approche de Djafarou : dataset Kaggle 5

4 Modèles de Classification **5**
4.1 CNN sur images brutes — Approche 1 (Christ Bohou) 5
4.1.1 Architecture 5
4.1.2 Résultats et analyse 6
4.2 Approche MLP et CNN sur images — Djafarou Oulare 7
4.2.1 MLP (Perceptron Multicouche) 7
4.2.2 CNN Kaggle 7
4.3 Réseau Dense sur Landmarks — Solution Retenue (Christ Bohou) . . . 8
4.3.1 Principe 8
4.3.2 Architecture 8
4.3.3 Résultats 8

5 Comparaison des Approches **9**
5.1 Analyse de la représentation 9

6 Intégration Temps Réel **9**
6.1 Architecture logicielle 10
6.2 Gestion du threading Python 10
6.3 Hold Timer dans le serveur Node.js 11
6.4 Cooldowns et seuil de confiance 11

7 Résultats et Analyse Critique **11**
7.1 Performance en conditions réelles 11
7.2 Limites identifiées 12

8 Perspectives d'Amélioration **12**
8.1 Court terme : commandes simultanées 12
8.2 Moyen terme : robustesse du geste FIRE 12

8.3 Long terme : architecture plus expressive 12

9 Conclusion **13**

A Structure du Projet **14**

B Guide de Lancement **14**

B.1 Prérequis 14

B.2 Lancement automatique 14

B.3 Lancement manuel 14

B.4 Test du modèle seul (sans jeu) 15

1 Introduction et Objectif

Ce projet s'inscrit dans le cadre du cours d'initiation à la Vision par Ordinateur du Master 2 Mathématiques. L'objectif est de concevoir un système complet de **reconnaissance de gestes de la main en temps réel** et de l'utiliser pour contrôler le jeu classique *Space Invaders* dans un navigateur web.

Le système repose sur une chaîne de traitement composée de quatre maillons :

1. **Capture vidéo** : webcam standard à 30 images/seconde.
2. **Détection de la main** : le modèle MediaPipe HandLandmarker extrait 21 points anatomiques en temps réel.
3. **Classification** : un réseau de neurones dense classifie le vecteur de 63 coordonnées normalisées en l'un des 5 gestes du jeu.
4. **Transmission** : les commandes sont envoyées via WebSocket (Node.js) au jeu JavaScript s'exécutant dans le navigateur.

Gestes reconnus — version finale

Geste	Description	Commande
	Main droite, index pointé vers la gauche	`LEFT`
	Main gauche, index pointé vers la droite	`RIGHT`
	Poing fermé (knuckles face caméra)	`FIRE`
	Main gauche, paume ouverte face caméra	`ENTER`
	Main au repos	`NEUTRAL`

2 Architecture Globale du Système

2.1 Vue d'ensemble

FIGURE 1 – Pipeline de contrôle gestuel temps réel.

2.2 Rôle de chaque composant

MediaPipe HandLandmarker. Modèle pré-entraîné de Google basé sur une architecture MobileNet légère. Il détecte la main avec BlazePalm puis prédit les coordonnées (x, y, z) de 21 points anatomiques (poignet, phalanges) en mode `VIDEO`, ce qui permet un suivi temporel efficace entre les images.

Réseau de neurones dense. Le vecteur de 63 coordonnées normalisées est classifié par un réseau entraîné sur nos propres données. Ce choix — travailler sur les *landmarks* plutôt que sur les pixels bruts — est la décision architecturale centrale du projet.

Serveur WebSocket Node.js. Un serveur `ws://localhost:8765` reçoit les commandes Python et les traduit en événements clavier pour le navigateur via un mécanisme de *hold timer* (180 ms) : la touche reste enfoncée tant que la commande arrive, puis se lève automatiquement.

3 Collecte et Préparation des Données

3.1 Approche 1 : images brutes (64×64 pixels)

La première tentative a consisté à collecter directement des **images RGB** de 64×64 pixels pour chaque geste. Le script `1_collecter_images.py` capture automatiquement une image toutes les 300 ms pendant 3 à 4 minutes par classe.

Classe	Nombre d'images
LEFT	502
RIGHT	516
FIRE	520
ENTER	520
NEUTRAL	463
Total	**2 521**

3.2 Approche 2 : landmarks normalisés (vecteur 63D)

La seconde approche — retenue — extrait les 21 landmarks MediaPipe et les normalise en un vecteur de 63 valeurs réelles.

Normalisation. Pour chaque frame, le vecteur brut $\mathbf{p} \in \mathbb{R}^{21\times 3}$ est transformé :

$$\tilde{\mathbf{p}} = \frac{\mathbf{p} - \mathbf{p}_0}{\max_i |\mathbf{p}_i - \mathbf{p}_0|} \tag{1}$$

où $\mathbf{p}_0$ est la position du poignet (point 0). Cette normalisation rend la représentation invariante à la **position** de la main dans le cadre, à l'**échelle** (distance à la caméra), et à la **taille** de la main.

Le vecteur aplati $\tilde{\mathbf{p}} \in [-1, 1]^{63}$ est sauvegardé en CSV avec sa classe. Environ 2 100 échantillons ont été collectés à raison de 1 capture automatique toutes les 300 ms.

3.3 Approche de Djafarou : dataset Kaggle

En parallèle, une seconde approche a été explorée sur un dataset externe Kaggle (*Hand Navigation Landmarks*, harisudhan411) contenant 8 000 images 180×180 en 4 classes (left, right, up, down) sur fond contrôlé uniforme.

Ensemble	**Train**	**Validation**	**Test**
Nombre d'images	8 000	400	1 200

4 Modèles de Classification

4.1 CNN sur images brutes — Approche 1 (Christ Bohou)

4.1.1 Architecture

Un CNN à 3 blocs convolutifs a été construit avec augmentation de données en ligne (*data augmentation*) pour compenser la faible taille du dataset.

Bloc	Couche	Filtres	Sortie
Bloc 1	Conv2D 3×3 ReLU (same)	32	64×64×32
	BatchNormalization	—	64×64×32
	Conv2D 3×3 ReLU (same)	32	64×64×32
	MaxPooling2D 2×2	—	32×32×32
	Dropout(0.25)	—	32×32×32
Bloc 2	Conv2D 3×3 ReLU (same)	64	32×32×64
	BatchNormalization	—	32×32×64
	Conv2D 3×3 ReLU (same)	64	32×32×64
	MaxPooling2D 2×2	—	16×16×64
	Dropout(0.25)	—	16×16×64
Bloc 3	Conv2D 3×3 ReLU (same)	128	16×16×128
	BatchNormalization	—	16×16×128
	MaxPooling2D 2×2	—	8×8×128
	Dropout(0.25)	—	8×8×128
Tête dense	Flatten	—	8 192
	Dense(256) ReLU	—	256
	BatchNormalization	—	256
	Dropout(0.5)	—	256
	Dense(5) Softmax	—	5

Paramètres entraînables : 2 240 037.

Configuration d'entraînement.

- Optimiseur : Adam ($\eta = 10^{-3}$), perte : *sparse_categorical_crossentropy*
- Batch : 32, max 30 epochs ; split 80/20
- Augmentation : flip horizontal, rotation ±15%, zoom ±15%, luminosité ±20%, contraste ±20%
- Callbacks : EarlyStopping (patience=8), ReduceLROnPlateau (factor=0.5, patience=4)

4.1.2 Résultats et analyse

Résultat — CNN images brutes

Meilleure val_accuracy : 34.3% (époque 14 sur 22). Précision chance aléatoire : 20% (1 classe sur 5).

Le modèle n'apprend pas à généraliser. L'accuracy de validation stagne autour de 20%

pendant la majeure partie de l'entraînement avec un pic isolé à 34%. **Cause principale** : le CNN apprend le fond de la pièce autant que la main elle-même. Avec seulement 2 521 images collectées dans un seul environnement, le modèle sur-apprend les artefacts de fond. La moindre variation d'éclairage ou de position dégrade les performances.

4.2 Approche MLP et CNN sur images — Djafarou Oulare

4.2.1 MLP (Perceptron Multicouche)

Couche	Unités
Flatten (180×180×3)	97 200
Dense ReLU	128
Dense Softmax	4

Entraîné sur 100 epochs avec Adam et *categorical_crossentropy*. Les données sont normalisées pixel/255 et encodées en one-hot.

4.2.2 CNN Kaggle

Couche	Sortie
Rescaling(1/255)	180×180×3
Conv2D(32, 3×3) ReLU	178×178×32
MaxPooling2D 2×2	89×89×32
Conv2D(64, 3×3) ReLU	87×87×64
MaxPooling2D 2×2	43×43×64
Flatten	118 336
Dense(128) ReLU	128
Dropout(0.5)	128
Dense(4) Softmax	4

Paramètres : 15 167 044. Ce modèle atteint 100% de val_accuracy sur le dataset Kaggle (fond uniforme) mais **échoue en conditions réelles** (webcam dans une pièce normale) en raison de son adaptation totale au fond contrôlé du dataset.

Limites des approches pixel

Les deux approches image-based (CNN et MLP) souffrent du même problème : elles apprennent le contexte visuel de fond autant que la forme de la main. Un dataset entraîné dans un bureau ne fonctionne pas dans une autre pièce.

4.3 Réseau Dense sur Landmarks — Solution Retenue (Christ Bohou)

4.3.1 Principe

Plutôt que de traiter les pixels, on extrait d'abord les 21 landmarks MediaPipe (représentation géométrique de la main), puis on entraîne un réseau dense sur ces coordonnées normalisées. La représentation est **invariante au fond, à la luminosité, à la couleur de peau et à la taille de la main**.

4.3.2 Architecture

Couche	Unités	Activation
Input	63	—
Dense	128	ReLU
BatchNorm	128	—
Dropout	128	— (taux : 0.3)
Dense	128	ReLU
BatchNorm	128	—
Dropout	128	— (taux : 0.3)
Dense	64	ReLU
BatchNorm	64	—
Dropout	64	— (taux : 0.2)
Dense	5	Softmax

Paramètres entraînables : 33 925 — soit **66 fois moins** que le CNN images.

Configuration d'entraînement. Optimiseur Adam ($\eta = 10^{-3}$), perte sparse_categorical_crossentr
batch 32, max 50 epochs (EarlyStopping patience=15), ReduceLROnPlateau.

4.3.3 Résultats

Résultat — Réseau dense sur landmarks

Meilleure val_accuracy : 99.6% (époque 13 sur 28).
2 erreurs sur 501 échantillons de validation.
Temps d'entraînement : $<$ 30 secondes sur CPU.

Matrice de confusion (501 échantillons de validation).

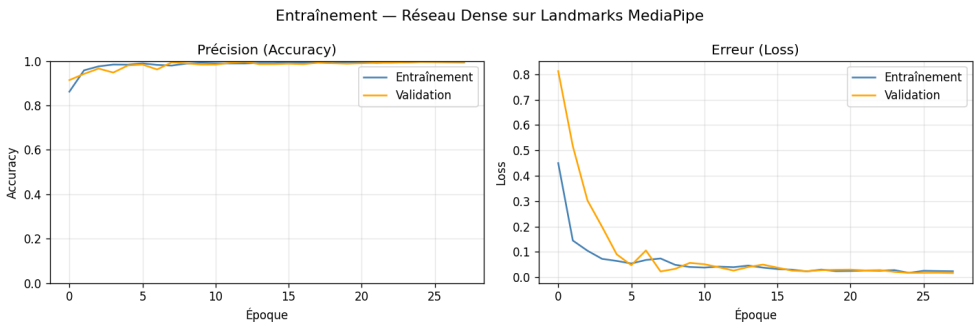

FIGURE 2 – Courbes d'entraînement du réseau dense sur landmarks. La convergence est rapide et stable.

Réel \ Prédit	ENTER	FIRE	LEFT	NEUTRAL	RIGHT
ENTER	**100**	0	0	0	0
FIRE	0	**100**	0	0	0
LEFT	0	0	**100**	0	0
NEUTRAL	0	1	1	**99**	0
RIGHT	0	0	0	0	**100**

Les 2 erreurs proviennent de la classe NEUTRAL, confondue une fois avec FIRE et une fois avec LEFT. Ceci s'explique géométriquement : une main en position de repos partielle ressemble à un poing ou à un index semi-étendu.

5 Comparaison des Approches

5.1 Analyse de la représentation

Le facteur déterminant de succès est le **choix de la représentation de l'entrée**, pas la complexité du modèle.

En passant des pixels bruts ($64 \times 64 \times 3 = 12\,288$ dimensions, sensibles au fond) aux landmarks normalisés (63 dimensions, invariants au fond), on :

- réduit la dimensionnalité d'un facteur **195**,
- élimine toute information de fond, de couleur et de taille,
- réduit les paramètres du modèle d'un facteur **66**,
- améliore la val_accuracy de **34.3**% → **99.6**%.

Cette observation est un exemple concret du principe général en apprentissage automatique : *une bonne représentation vaut mieux qu'un grand modèle.*

6 Intégration Temps Réel

Critère	CNN images	MLP Kaggle	CNN Kaggle	Dense landmarks
Val. accuracy	34.3%	N/A	∼100%	**99.6%**
Perf. en conditions réelles	Mauvaise	Mauvaise	Mauvaise	**Bonne**
Paramètres	2.24M	12.5M	15.2M	**33 925**
Temps/epoch (CPU)	∼16 s	∼60 s	∼45 s	**$<$ 1 s**
Données nécessaires	50 000+	8 000 (ext.)	8 000 (ext.)	**∼2 000**
Sensibilité au fond	Critique	Critique	Critique	**Nulle**
Invariant taille main	Non	Non	Non	**Oui**
Dataset propre	Oui	Non (Kaggle)	Non (Kaggle)	**Oui**

TABLE 1 – Comparaison des quatre approches expérimentées.

6.1 Architecture logicielle

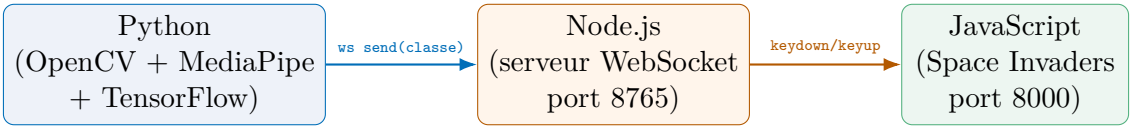

FIGURE 3 – Architecture trois processus du système de contrôle gestuel.

6.2 Gestion du threading Python

Le script `5_jouer_avec_cnn.py` utilise une architecture bi-thread pour éviter le blocage réseau :

- **Thread principal** : capture OpenCV + inférence MediaPipe + CNN (traitement synchrone, 30 fps)
- **Thread WebSocket** : lecture de la file `Queue` et envoi des commandes (non-bloquant grâce à `put_nowait`)

Listing 1 – Communication inter-threads par file de messages.

```
commande_queue = queue.Queue(maxsize=20)

# Thread principal : rempli la file
commande_queue.put_nowait({'type': geste})
```

```

# Thread WS : vide la file
msg = commande_queue.get(timeout=0.1)
ws.send(json.dumps(msg))
```

6.3 Hold Timer dans le serveur Node.js

Le jeu JavaScript génère des événements clavier (`keydown`/`keyup`). Or Python envoie des commandes discrètes (à intervalle irrégulier), pas des états continus. Le serveur Node.js résout ce découplage avec un **hold timer** de 180 ms :

- À la réception de `LEFT`, le serveur envoie `keydown(37)`.
- Si aucune nouvelle commande `LEFT` n'arrive dans les 180 ms, il envoie `keyup(37)`.
- Chaque nouvelle commande réinitialise le timer, maintenant la touche enfoncée tant que le geste est détecté.

Ce mécanisme permet un mouvement fluide continu même avec une reconnaissance à 30 fps.

6.4 Cooldowns et seuil de confiance

Pour éviter les faux positifs et les commandes parasites :

Commande	Cooldown	Seuil confiance
LEFT	80 ms	70%
RIGHT	80 ms	70%
FIRE	80 ms	70%
ENTER	800 ms	70%
NEUTRAL	— (jamais envoyé)	—

7 Résultats et Analyse Critique

7.1 Performance en conditions réelles

Le réseau dense sur landmarks atteint **99.6% en validation** et **fonctionne de manière fiable en temps réel** sur webcam dans des conditions d'éclairage variées. Les gestes LEFT, RIGHT, ENTER et NEUTRAL sont reconnus sans erreur notable. Le geste FIRE (poing fermé) présente une légère ambiguïté quand la main est semi-fermée (transition vers NEUTRAL).

7.2 Limites identifiées

1. **FIRE entraîné sur une seule main.** Les données de collecte ont été capturées principalement avec la main gauche. Le modèle reconnaît moins bien le même geste effectué avec la main droite, car les landmarks sont symétriques (coordonnées x inversées) et constituent une forme différente pour le réseau.
2. **Impossibilité de tirer en se déplaçant.** La classification étant unimodale (une classe par frame), le système ne peut envoyer qu'une commande à la fois. Il est impossible de prédire simultanément LEFT et FIRE.
3. **Latence** : la chaîne complète (capture → MediaPipe → CNN → WebSocket → jeu) introduit environ 40 à 80 ms de latence, imperceptible en gameplay normal.
4. **Dépendance à MediaPipe.** Si la main sort du cadre ou est partiellement occultée, MediaPipe ne détecte rien et aucune commande n'est envoyée (comportement sûr mais parfois gênant).

8 Perspectives d'Amélioration

8.1 Court terme : commandes simultanées

La limitation la plus impactante sur la jouabilité est l'impossibilité de tirer en se déplaçant. Une solution est d'ajouter 2 classes composites :

Classe	Geste	Commandes envoyées
`LEFT_FIRE`	Poing incliné à gauche	LEFT + FIRE
`RIGHT_FIRE`	Poing incliné à droite	RIGHT + FIRE

La transition `LEFT` → `LEFT_FIRE` consiste à fermer le poing sans bouger le bras — geste naturel et peu fatigant.

8.2 Moyen terme : robustesse du geste FIRE

Collecter le geste FIRE avec **les deux mains** à parts égales. Les landmarks de la main droite étant le miroir de la gauche (coordonnée $x \to -x$), on peut aussi faire de l'augmentation de données par réflexion sans recollecte.

8.3 Long terme : architecture plus expressive

— **LSTM ou Transformer temporel** : encoder non plus une frame mais une séquence de 5 à 10 frames de landmarks. Cela permettrait de reconnaître des *gestes dynamiques* (main qui se déplace vers la droite, pas juste pointée à droite) et d'éliminer les faux positifs de transition.

— **Deux mains simultanées** : MediaPipe supporte jusqu'à 2 mains. En traitant les deux, on pourrait par exemple : main gauche = direction, main droite = tir, ce qui résout le problème de simultanéité de manière naturelle.

9 Conclusion

Ce projet a permis de concevoir et déployer un pipeline complet de reconnaissance de gestes en temps réel, depuis la capture webcam jusqu'au contrôle d'un jeu dans le navigateur. Il illustre plusieurs concepts fondamentaux de la vision par ordinateur et de l'apprentissage automatique :

— **L'importance de la représentation** : le choix des landmarks normalisés plutôt que des pixels bruts a été le facteur décisif, faisant passer la précision de 34.3% à 99.6% avec un modèle 66 fois plus petit.
— **La généralisation** : un modèle entraîné sur un dataset externe (Kaggle, fond contrôlé) ne transfère pas en conditions réelles. Nos propres données, bien que moins nombreuses, donnent un modèle qui fonctionne dans notre environnement.
— **L'ingénierie système** : la performance perçue dépend autant du modèle que de l'architecture (threading, WebSocket, hold timer). Un bon modèle mal intégré donne une mauvaise expérience utilisateur.
— **Les limites honnêtes** : le système ne permet pas encore de tirer en se déplaçant, et FIRE est moins robuste avec la main droite. Ces limites sont comprises et documentées, avec des solutions identifiées.

Le code source complet est disponible sur : `https://github.com/bohouchrist/projet_computer_vision`

A Structure du Projet

```
projet_computer_vision/
|-- index.html                  # Jeu Space Invaders
|-- comment_jouer.html          # Guide des gestes (visuel interactif)
|-- server.js                   # Serveur WebSocket Node.js
|-- lancer_jeu.ps1              # Script de demarrage automatique
|-- hand_landmarker.task        # Modele MediaPipe pre-entraine
|-- game.bundle.js              # Jeu compile
|-- core/ entities/ utils/      # Code source du jeu
+-- cnn_training/
    |-- 1_collecter_images.py       # Collecte images brutes
    |-- 2_entrainer_cnn.py          # CNN sur images (approche 1)
    |-- 3_collecter_landmarks.py    # Collecte landmarks MediaPipe
    |-- 4_entrainer_landmarks.py    # Reseau dense sur landmarks
    |-- 5_jouer_avec_cnn.py         # Script de jeu (solution finale)
    |-- test_cnn_local.py           # Test temps reel sans jeu
    +-- modeles/
        |-- gestes_landmarks.keras  # Modele final (99.6%)
        |-- classes_landmarks.txt   # Liste des 5 classes
        |-- courbes_landmarks.png   # Courbes d'entrainement
        +-- images_rapport/         # Photos des gestes
```

B Guide de Lancement

B.1 Prérequis

```
pip install tensorflow mediapipe opencv-python websocket-client
npm install  # dans le dossier du projet
```

B.2 Lancement automatique

```
# Depuis PowerShell, dans le dossier du projet :
.\lancer_jeu.ps1
```

Ce script ouvre automatiquement les 3 processus (WebSocket, HTTP, jeu) et lance la détection webcam dans le terminal courant.

B.3 Lancement manuel

```
# Terminal 1 - Serveur WebSocket
```

```
node server.js

# Terminal 2 - Serveur HTTP du jeu
python -m http.server 8000

# Terminal 3 - Détection et contrôle
cd cnn_training
python 5_jouer_avec_cnn.py

# Navigateur
http://localhost:8000
```

B.4 Test du modèle seul (sans jeu)

```
cd cnn_training
python test_cnn_local.py
```